MACHINE LEARNING IN PHYSICS
CONVOLUTIONAL NEURAL NETWORKS

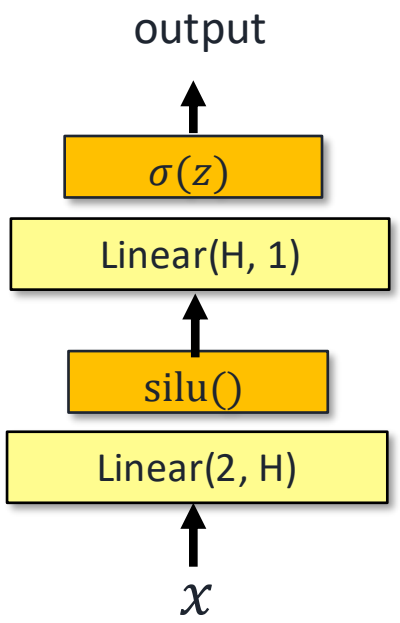
HARRISON B. PROSPER

PHY6937 / PHY4936

Recap

We used the simplest machine learning model, namely a feed-forward neural network (FFNN), such as the one shown here, to review a few key concepts in ML, including

1. the loss function, L,
2. the empirical risk (i.e., average loss), $R(\omega)$, the risk functional, $R[f]$, the equation $\mathbb{E}_{p(y \mid x)}\left[\frac{\partial L}{\partial f}\right] = 0$,
3. and empirical risk minimization via stochastic gradient descent.



Plan

The plan for the next few lectures is to introduce a few models and methods from the following list:

1. Convolutional neural networks (CNN)
2. Physics-informed neural networks (PINN)
3. Graph neural networks (GNN)
4. Autoencoders (AE)
5. Flow and diffusion models
6. Transformer neural networks (TNN)
7. LLM-based agentic AI
8. Bayesian neural networks
9. Simulation-based inference

Outline

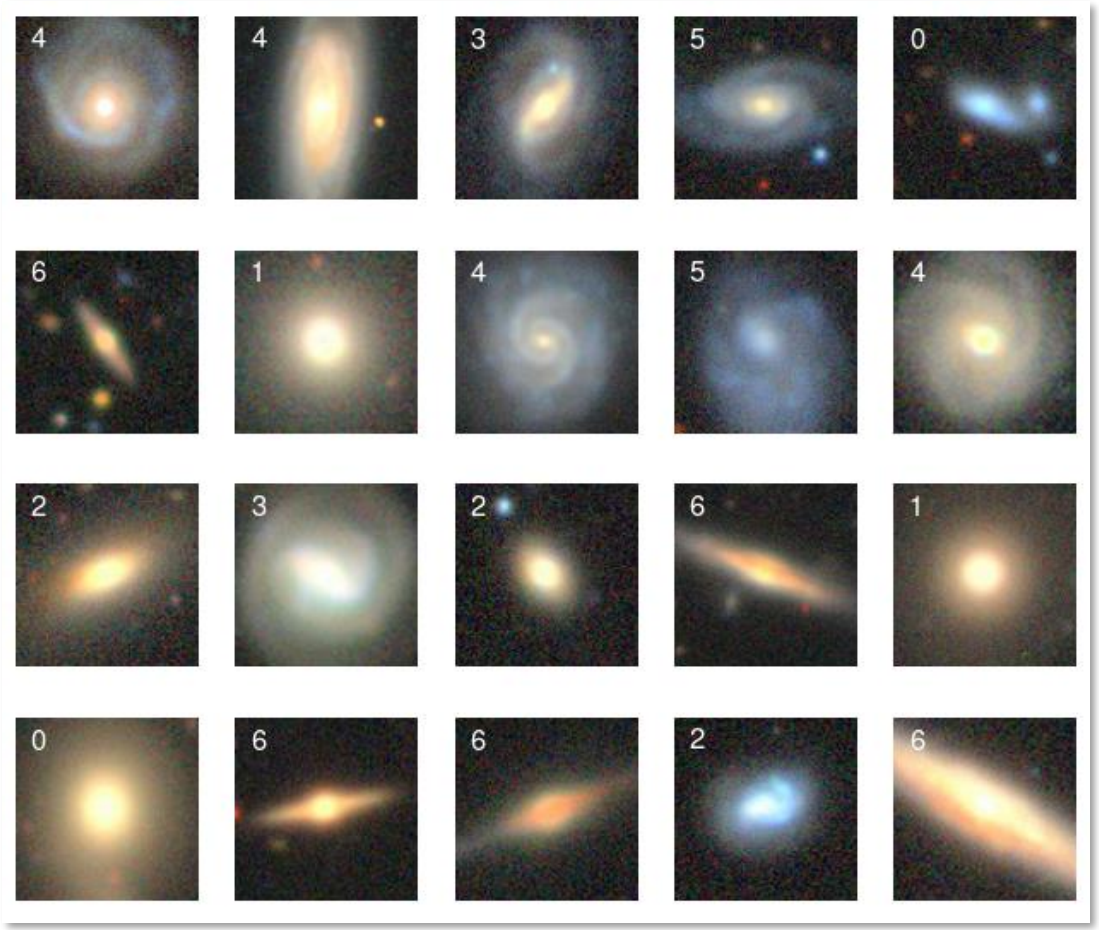
- A Galaxy Classifier: Part 1
  - Multi-Class Classification
  - Convolutional Neural Networks
- A Galaxy Classifier: Part 2
- Summary
- Appendix A, B

A GALAXY CLASSIFIER: PART 1

A Galaxy Classifier (1)

Goal: classify galaxies using images like these.

The images are from the Galaxy 10 DECals Dataset at the astroNN website.

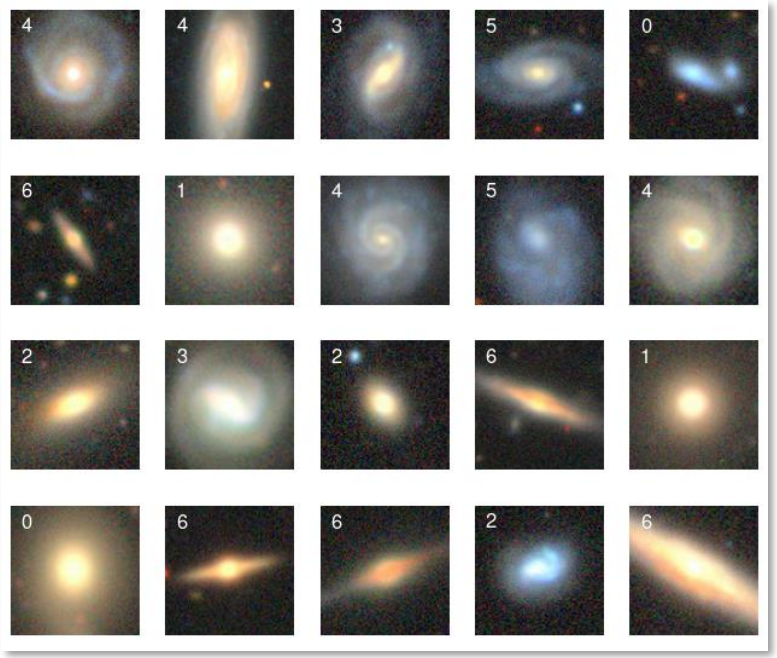


https://astronn.readthedocs.io/en/stable/galaxy10.html

A Galaxy Classifier (2)

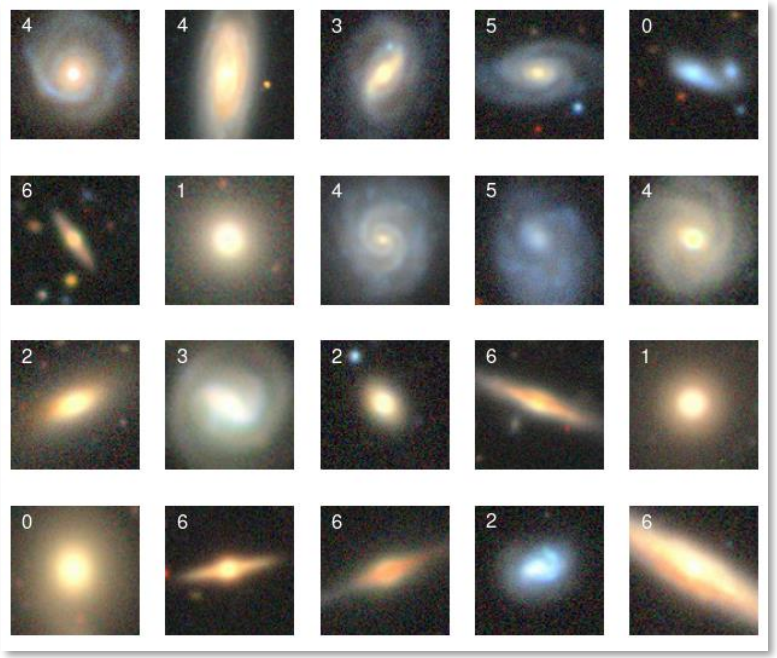
Dataset

1. The dataset comprises 12,600 images divided into 7 galaxy morphology classes with 1800 galaxies per class.
2. Each 3-channel image is cropped to 96 x 96 pixels.



A Galaxy Classifier (3)

This a **multi-class** problem, so we need a generalization of binary classification which reduces to the latter when there are 2 classes.



MULTI-CLASS CLASSIFICATION

Multi-Class Classification (1)

In the galaxy classification problem, we have $K = 7$ classes.

Therefore, we need K functions, $f_k(x)$, that satisfy

$$\sum_{k=0}^{K-1} f_k(x) = 1, \qquad \forall x$$

These functions should approximate the class probabilities,

$$f_k(x) \approx p(k \mid x) = \frac{p(x \mid k)\,\pi(k)}{p(x)}, \qquad \sum_{k=0}^{K-1} \pi(k) = 1$$

Multi-Class Classification (2)

Given a loss function $L(y, f)$ we can, in principle, determine what a trained model $f_\omega(x)$ approximates.

But now we wish to go in the reverse direction:

Given $f_k \equiv f_{k,\omega}(x) \approx p(k \mid x)$ what loss function $L(y, f)$ should one use?

The question can be answered by minimizing the risk functional $R[f]$, but this time we must impose the constraint:

$$\sum_{k=0}^{K-1} f_k = 1\,, \qquad \forall x$$

Multi-Class Classification (3)

Recall that the risk functional can be written as

$$R[f] = \mathbb{E}_{p(x)}[\mathcal{L}(x, f)]$$

with

$$\mathcal{L}(x, f) = \mathbb{E}_{p(y \mid x)}[L(y, f)]$$

But since the values of y are discrete: $y = 0, \cdots, K-1$,

$$\mathcal{L}(x, f) = \sum_{k=0}^{K-1} L_k \, p_k$$

where $L_k \equiv L(k, f_k)$ and $p_k \equiv p(k \,|x)$.

Multi-Class Classification (4)

We want to minimize

$$R[f] = \mathbb{E}_{p(x)}\left[\sum_{k=0}^{K-1} L_k \, p_k\right]$$

with respect to f_k, subject to the constraint

$$\sum_{k=0}^{K-1} f_k = 1\,, \qquad \forall x$$

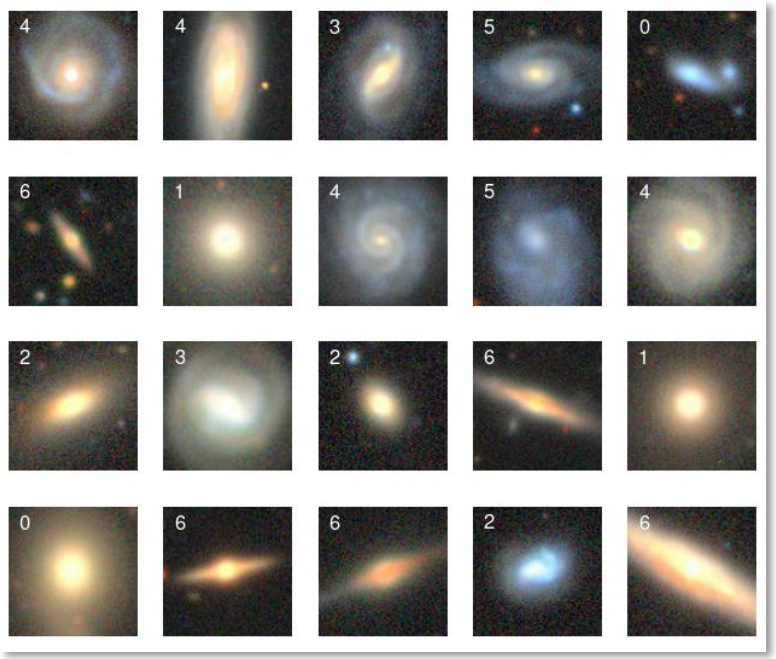
that is, subject to

$$\mathbb{E}_{p(x)}\left[\sum_{k=0}^{K-1} f_k\right] - 1 = 0$$

Multi-Class Classification (5)

The solution (see Appendix B) is the cross-entropy loss

$$L_k = -\log f_k.$$



Therefore, we can solve the multi-class problem by minimizing the following empirical risk

$$R(\omega) = \frac{1}{N}\sum_{i=0}^{N-1}[-\log f_{y_i,\omega}(x_i)]$$

where $k = y_i$ is the known class index of image i and, $f_{k,\omega}$, is a model that we hope is flexible enough to approximate K probabilities that sum to one.

CONVOLUTIONAL NEURAL NETWORKS (CNN)

Convolutional Neural Networks (1)

- Convolutional neural networks (CNN) are functions that extract features from 1, 2 or 3D data, and then process them with another neural network.

- The insight that underpins CNNs is that many data sets, for example images, have similar patterns in different parts of the data sets.



Harrison Prosper, Dinan, France, 2025

Convolutional Neural Networks (2)

A standard CNN comprises three types of processing layers:

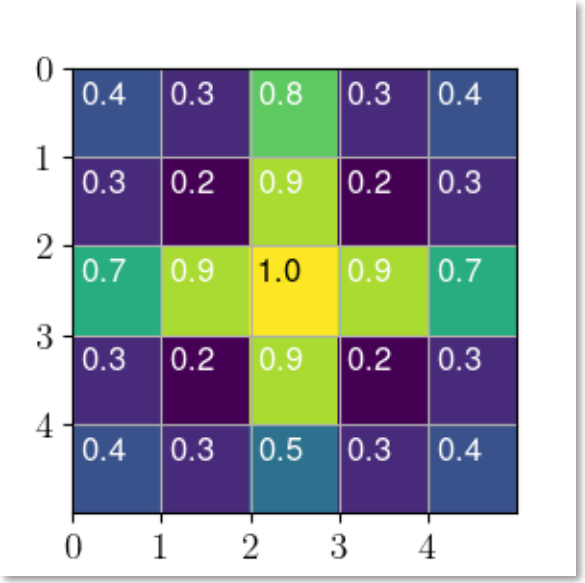
1. **Convolution**
   The data, e.g., an image, is *convolved* with one or more filters.

2. **Down-sampling**
   After applying a nonlinear map to the convolved data, the latter are down-sampled.

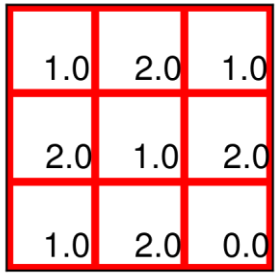
3. **Classification**
   A feed-forward neural network classifies the convolved data.

Convolutional Neural Networks (3)

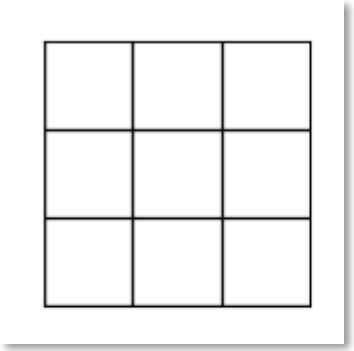
1. Convolution



(5, 5, 1)-image



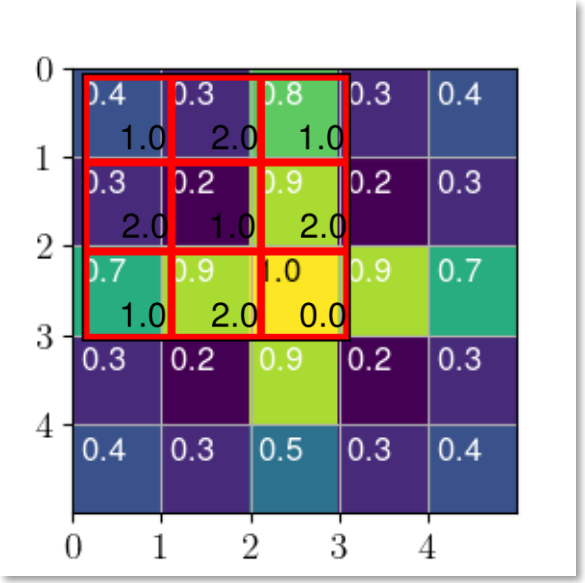
(3, 3)-filter



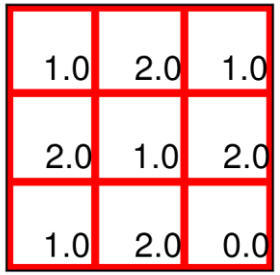
(3, 3, 1)-image

Convolutional Neural Networks (4)

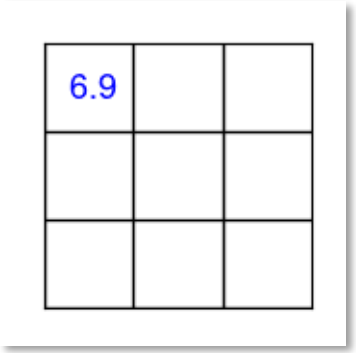
1. Convolution



(5, 5, 1)-image



(3, 3)-filter



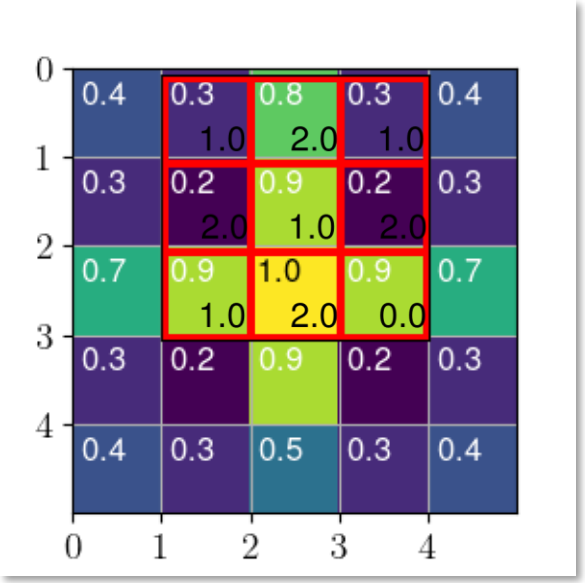
(3, 3, 1)-image convolved

stride = 1

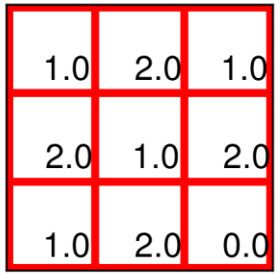
$$\mathbf{0.4}\times 1.0 + \mathbf{0.3}\times 2.0 + \mathbf{0.8}\times 1.0 + \mathbf{0.3}\times 2.0 + \mathbf{0.2}\times 1.0 + \mathbf{0.9}\times 2.0 + \mathbf{0.7}\times 1.0 + \mathbf{0.9}\times 2.0 + \mathbf{1.0}\times 0.0 = 6.9$$

Convolutional Neural Networks (5)

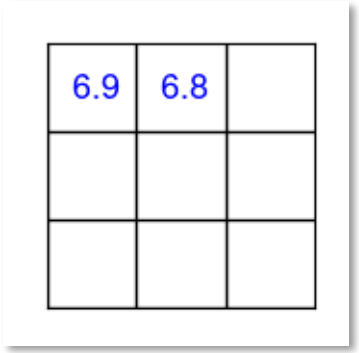
1. Convolution



(5, 5, 1)-image



(3, 3)-filter

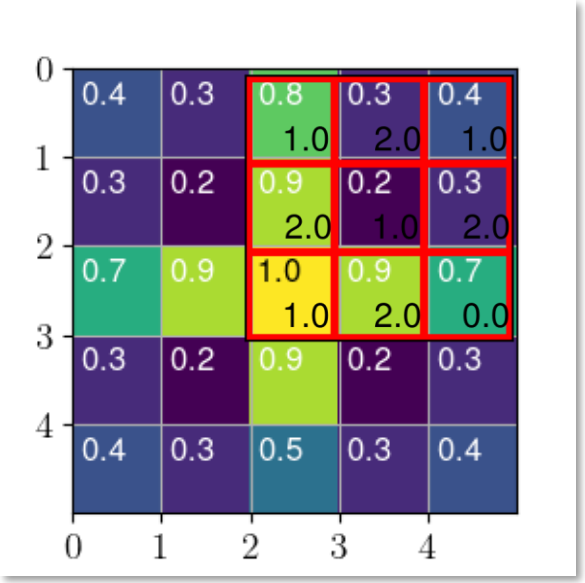


(3, 3, 1)-image convolved

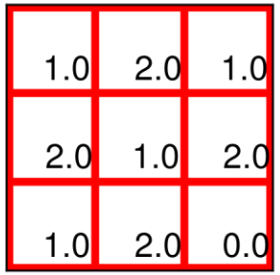
$$\mathbf{0.3}\times 1.0 + \mathbf{0.8}\times 2.0 + \mathbf{0.3}\times 1.0 + \mathbf{0.2}\times 2.0 + \mathbf{0.9}\times 1.0 + \mathbf{0.2}\times 2.0 + \mathbf{0.9}\times 1.0 + \mathbf{1.0}\times 2.0 + \mathbf{0.9}\times 0.0 = 6.8$$

Convolutional Neural Networks (6)

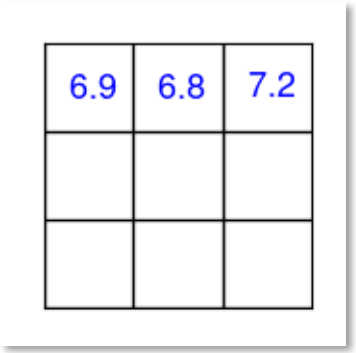
1. Convolution



(5, 5, 1)-image



(3, 3)-filter



(3, 3, 1)-image

(convolved)

$$\mathbf{0.8}\times 1.0 + \mathbf{0.3}\times 2.0 + \mathbf{0.4}\times 1.0 +$$
$$\mathbf{0.9}\times 2.0 + \mathbf{0.2}\times 1.0 + \mathbf{0.3}\times 2.0 +$$
$$\mathbf{1.0}\times 1.0 + \mathbf{0.9}\times 2.0 + \mathbf{0.7}\times 0.0 = 7.2$$

Convolutional Neural Networks (6.1)

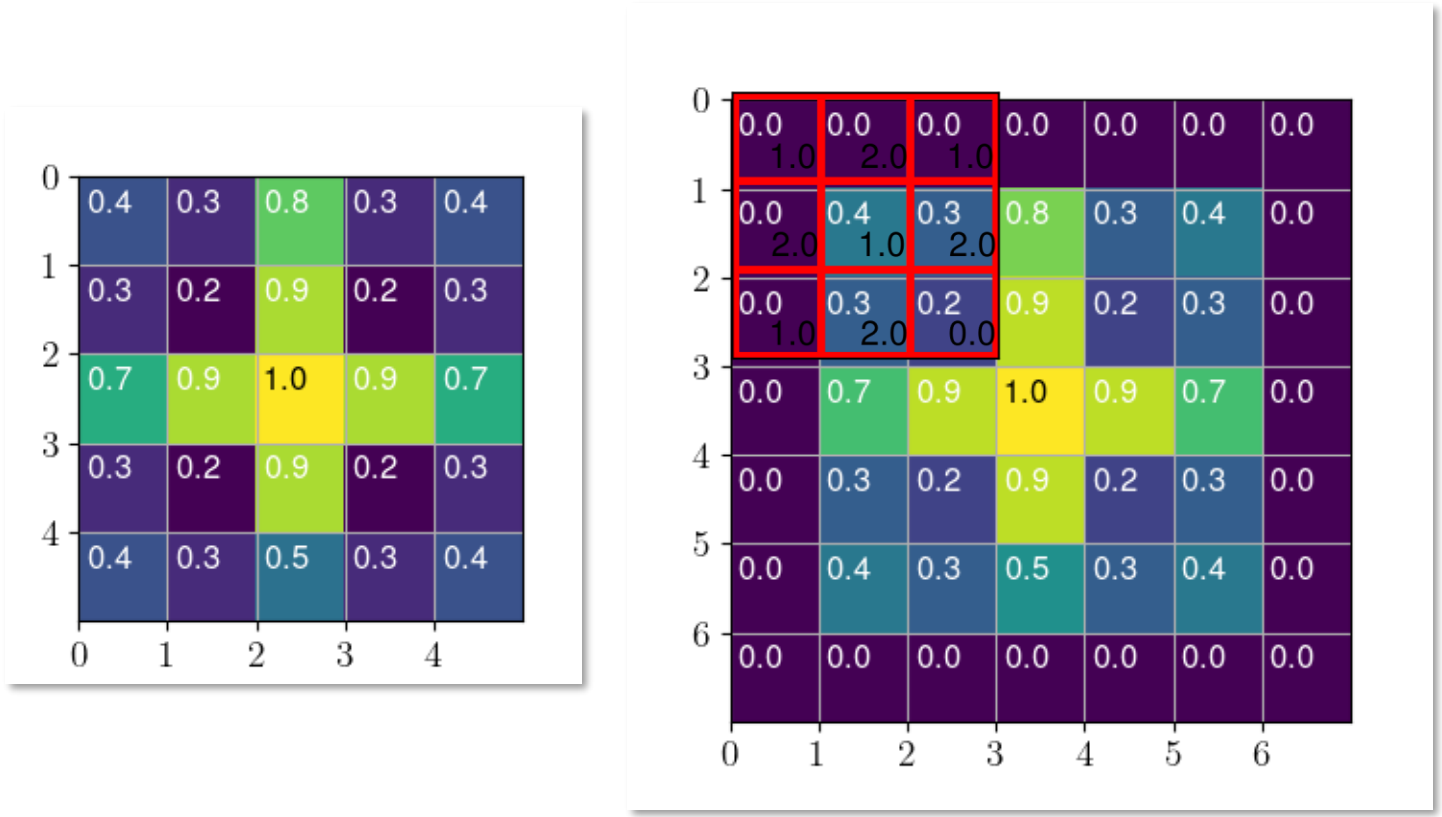
The matrix elements of a filter are the parameters to be fitted. In addition, there is one bias parameter per output channel.

- A convolution that processes an input image with C_{in} channels and outputs another with C_{out} channels has $C_{out} \times C_{in}$ separate filters, each of which creates a single-channel image, $I_{n,m}$, indexed by a pair of integers (n, m) where $0 \leq n \leq C_{out} - 1$ and $0 \leq m \leq C_{in} - 1$. Therefore, $C_{out} \times C_{in}$ single-channel images are created.

- The final step is a pixel-by-pixel sum, $I_n = \sum_m I_{n,m}$, over the input channel index m, thereby yielding an output image with C_{out} channels.

Convolutional Neural Networks (7)

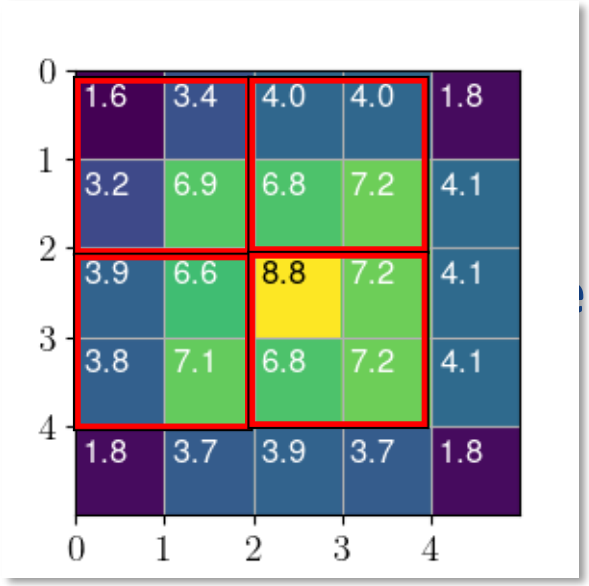
Padding



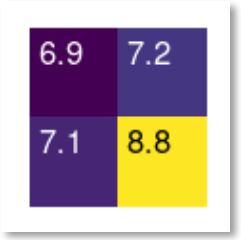
Wrapping the image with a 1–pixel border makes it possible, with a (3, 3)-filter, to maintain the image size, at the cost potentially of diluting the information in the image.

Convolutional Neural Networks (8)

2. Down-sampling



= 2



(2, 2, 1)-image

(down-sampled)

(5, 5, 1)-image (convolved)

MaxPool2d Return the largest value within window.

AvgPool2d Return the average value within window.

Convolutional Neural Networks (9)

3. Classification

The convolved data go to a feed-forward neural network whose outputs, z_j, go through a softmax function

$$f_k = \text{softmax}(x) \equiv \frac{e^{z_k}}{\sum_j e^{z_j}}$$

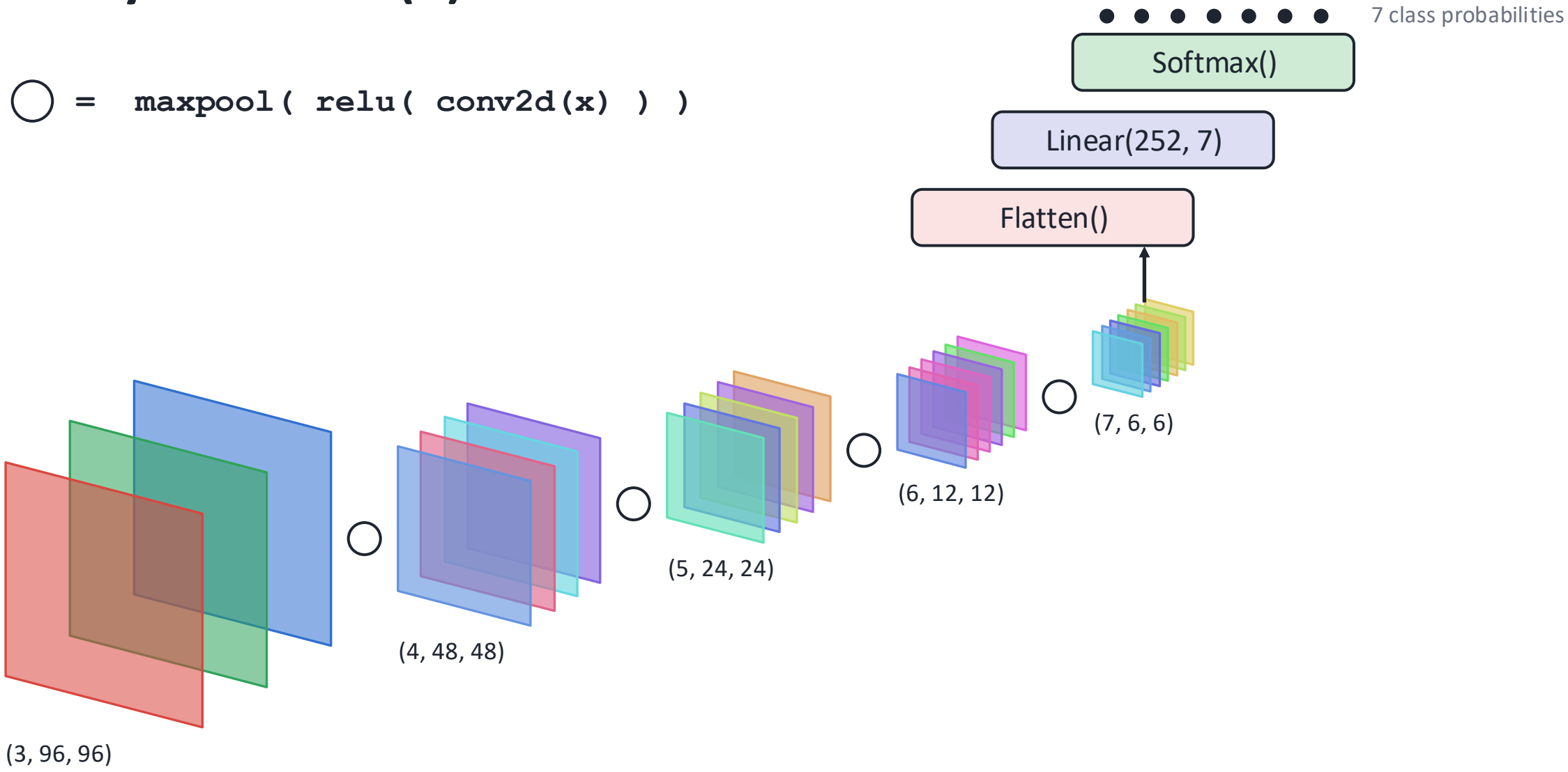
that enforces the sum rule

$$\sum_{k=0}^{K-1} f_k = 1$$

A GALAXY CLASSIFIER: PART 2

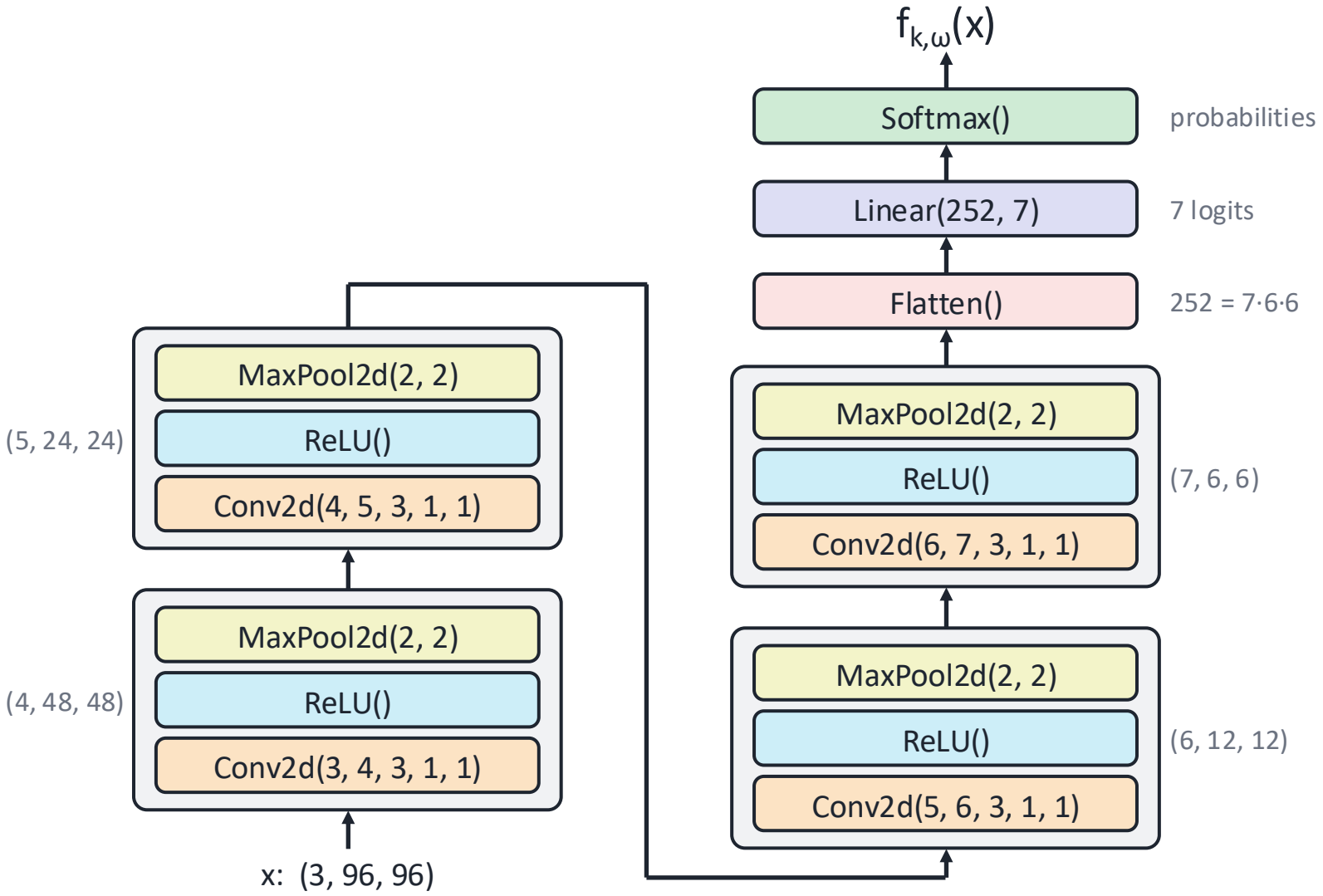
A Galaxy Classifier (4)

◯ = `maxpool( relu( conv2d(x) ) )`



A Galaxy Classifier (5)

Here is a more standard representation of the proposed classifier.



Summary

- Convolutional neural networks have revolutionized image classification and interpretation and are now used in many applications.
- The model exploits the fact that natural images are often locally approximately translation invariant.
- The convolutions automatically extract features from images that can later be used to interpret them.

APPENDIX A

Method of Lagrange Multipliers (1)

An elegant way to account for a constraint in a minimization task is the method of **Lagrange* multipliers**. Consider the following classic problem: Find the coefficients c_i in the sum

$$F = \sum_{i=0}^{K-1} c_i \, x_i, \qquad x_i \sim p(x)$$

subject to the constraint

$$\sum_{i=0}^{K-1} c_i = 1$$

that minimizes the variance of F.

*Joseph-Louis Lagrange (1736 – 1813)

Method of Lagrange Multipliers (2)

- Step 1: Find a formula for the variance of F

$$F = \sum_{i=0}^{K-1} c_i \, x_i \quad \Longrightarrow \quad \Delta F = \sum_{i=0}^{K-1} c_i \, \Delta x_i$$

$$\mathbb{V}[F] \equiv \mathbb{E}_{p(x)}[(\Delta F)^2] = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} c_i c_j \mathbb{E}_{p(x)}\left[\Delta x_i \Delta x_j\right]$$

Assuming the x_i are statistically independent, we obtain

$$\mathbb{V}[F] = \sum_{i=0}^{K-1} c_i^2 \, \mathbb{V}_i$$

where $\mathbb{V}_i = \mathbb{E}_{p(x)}[(\Delta x_i)^2]$.

Method of Lagrange Multipliers (3)

- Step 2: Add the constraint to the variance $\mathbb{V}[F]$ using a Lagrange multiplier λ:

$$\mathbb{V}[F] = \sum_{i=0}^{K-1} c_i^2 \, \mathbb{V}_i + \boldsymbol{\lambda}\left(\sum_{\boldsymbol{i=0}}^{\boldsymbol{K-1}} \boldsymbol{c_i} \; \boldsymbol{-1}\right)$$

- Step 3: Minimize $\mathbb{V}[F]$ with respect to c_j

$$\frac{\partial \mathbb{V}}{\partial c_j} = 2c_j \mathbb{V}_j + \lambda = 0 \implies c_j = -\frac{\lambda}{2\mathbb{V}_j}$$

Method of Lagrange Multipliers (4)

- Step 4: To find the multiplier, λ, use the constraint

$$\sum_{i=0}^{K-1} c_i = 1, \qquad c_j = -\frac{\lambda}{2\mathbb{V}_j}$$

that is,

$$-\frac{\lambda}{2}\sum_{i=0}^{K-1}\frac{1}{\mathbb{V}_i} = 1$$

- Therefore, the variance of $F = \sum_{i=0}^{K-1} c_i\, x_i$ is minimized when

$$c_j = \frac{1}{\mathbb{V}_j} \Big/ \sum_{i=0}^{K-1}\frac{1}{\mathbb{V}_i}$$

APPENDIX B

Cross Entropy Loss (1)

The risk functional can be written as

$$R[f] = \mathbb{E}_{p(x)}[\mathcal{L}(x, f)]$$

with

$$\mathcal{L}(x, f) = \mathbb{E}_{p(y \mid x)}[L(y, f)]$$

But since the values of y are discrete: $y = 0, \cdots, K-1$,

$$\mathcal{L}(x, f) = \sum_{k=0}^{K-1} L_k \, p_k$$

where $L_k \equiv L(k, f_k)$ and $p_k \equiv p(k \,|x)$.

Cross Entropy Loss (2)

A perfect model, f_k, would satisfy $f_k = p_k$.

The question is: What loss function yields this result?

We saw in previous lectures that the optimal solution $f_k = p_k$ is found by minimizing the risk functional $R[f]$.

But this time, we need to account for the constraint:

$$\sum_{k=0}^{K-1} f_k = 1\,, \qquad \forall x$$

Cross Entropy Loss (3)

We'll use the method of Lagrange multipliers.

$$R[f] = \mathbb{E}_{p(x)}\left[\sum_{k=0}^{K-1} L_k\, p_k\right] + \boldsymbol{\lambda}\left(\mathbb{E}_{\boldsymbol{p(x)}}\left[\sum_{\boldsymbol{k=0}}^{\boldsymbol{K-1}} \boldsymbol{f_k} - \mathbf{1}\right]\right)$$

To obtain the minimum of $R[f]$ set

$$\frac{\delta R}{\delta f_j} = 0$$

This yields

$$\mathbb{E}_{p(x)}\left[\sum_{k=0}^{K-1}\left(\frac{\partial L_k}{\partial f_j}\, p_k + \lambda \frac{\partial f_k}{\partial f_j}\right)\right] = 0$$

Cross Entropy Loss (4)

From

$$\mathbb{E}_{p(x)}\left[\sum_{k=0}^{K-1}\left(\frac{\partial L_k}{\partial f_j}\,p_k+\lambda\frac{\partial f_k}{\partial f_j}\right)\right]=0$$

only the term with $k=j$ survives, yielding

$$\mathbb{E}_{p(x)}\left[\frac{\partial L_j}{\partial f_j}\,p_j+\lambda\right]=0$$

(Note: there is no sum over j!)

Cross Entropy Loss (5)

We want the expression

$$\mathbb{E}_{p(x)}\left[\frac{\partial L_j}{\partial f_j}\, p_j + \lambda\right] = 0$$

to hold for every value of x and for every density $p(x)$.

This will be true *if and only if*

$$\frac{\partial L_j}{\partial f_j}\, p_j + \lambda = 0$$

Since the optimal solution is $f_j = p_j$, this implies

$$\frac{\partial L_j}{\partial f_j}\, f_j + \lambda = 0$$

Cross Entropy Loss (6)

Solving

$$\frac{\partial L_j}{\partial f_j} f_j + \lambda = 0$$

for the loss, L_j, we find

$$L_j = -\lambda \log f_j$$

We can always rescale the loss so that $\lambda = 1$.

$L_j = -\log f_j$ is called the **cross-entropy loss**.